

Implementation and Evaluation of Quantum Algorithms for the Clique Problem

Alen Leban, Uroš Čibej

University of Ljubljana; Faculty of Computer and Information Science
Večna pot 113, 1000 Ljubljana, Slovenija
E-mail: al2719@student.uni-lj.si, uros.cibej@fri.uni-lj.si

Abstract

The Maximum Clique Problem is a well-known NP-hard combinatorial optimization problem with numerous practical applications. With advances in quantum computing, several quantum algorithmic paradigms offer promising avenues for solving such problems. In this paper, we present the implementation and evaluation of three distinct quantum approaches: the Quantum Approximate Optimization Algorithm (QAOA), Quantum Annealing (QA), and Grover's search algorithm. We analyze their structural advantages and disadvantages, focusing on solution accuracy, success probability, and resource consumption in terms of qubits and circuit depth.

1 Introduction

Clique problems are key in graph theory, aiming to find the largest or a complete subgraph of specific-size. These problems are widely applicable in social network analysis, bioinformatics, finance, and circuit design. Garey and Johnson [1] show that the decision version is NP-complete and the optimization version is NP-hard, making exact solutions for large instances difficult on classical computers, leading researchers to explore alternative paradigms.

Quantum computing offers several approaches to solving such combinatorial problems, including heuristic gate-based methods (QAOA [2]), analog optimization (QA), and exact search algorithms such as Grover's algorithm [3]. Although these three quantum paradigms are well-understood theoretically, their practical performance, resource needs, and trade-offs vary when applied to specific graph problems. Literature often analyzes these methods in isolation [4] or on broad benchmarks [5]. Cross-paradigm evaluations are vital to understand how graph features such as density, size, and constraints impact circuit depth, qubit usage, and success probability.

This paper presents a comparative implementation and evaluation of QAOA, QA, and Grover's algorithm for the clique problem. We map problems to quantum formulations, using Quadratic Unconstrained Binary Optimization (QUBO) for QAOA and QA, and a custom oracle for Grover's. Evaluation focuses on correct solution probability, and resource use measured by qubits and circuit depth.

The remainder of this paper is organized as follows. Section 2 establishes the mathematical preliminaries of the clique problem and provides an overview of the three quantum algorithmic paradigms. Section 3 details our specific mapping strategies and adaptations for the clique problem across each algorithm. Section 4 describes our empirical experimental setup and discusses the evaluation results. Finally, Section 5 concludes the paper and outlines avenues for future work.

2 Preliminaries

Given an undirected graph $G = (V, E)$, where V is the set of vertices (nodes) and E is the set of edges (connections), a clique is a subset of vertices $C \subseteq V$ such that every pair of distinct vertices in C is adjacent (connected by an edge). In other words, the subgraph induced by C is a complete graph.

In this paper, we will deal mostly with the decision and optimization versions of the problem - hereafter referred to as **KClique** and **MaxClique**, respectively.

The clique problems are computationally hard [1], under current knowledge, thus untractable even with quantum computers. However, a few quantum techniques show promising potential for practical speedups; three of these will be examined here.

2.1 Quantum Approximate Optimization Algorithm (QAOA)

The Quantum Approximate Optimization Algorithm, introduced by Farhi et al. [2], approximates adiabatic evolution by alternating applications of a mixer Hamiltonian H_M and a problem Hamiltonian H_P , implemented as a variational quantum circuit whose parameters are optimized by a classical optimizer. While the mixer Hamiltonian promotes exploration of the search space, the problem Hamiltonian is constructed such that its ground state corresponds to the optimal solution of the combinatorial problem, commonly formulated as QUBO. The objective function is expressed using binary decision variables $x_i \in \{0, 1\}$, which are then mapped to qubit operators using the transformation $x_i = \frac{1-Z_i}{2}$, where Z_i denotes the Pauli-Z operator acting on qubit i . Since the eigenvalues of Z_i are $+1$ and -1 , this transformation allows the QUBO objective function to be rewritten as a Hamiltonian composed of Pauli-Z terms whose ground state encodes the

optimal solution.

2.2 Quantum Annealing

Quantum Annealing is a quantum optimization technique closely related to Adiabatic Quantum Computation (AQC). The system evolves from an initial Hamiltonian (usually the transverse-field Hamiltonian) to a problem Hamiltonian whose ground state encodes the solution. According to the adiabatic theorem, sufficiently slow evolution keeps the system in the ground state throughout the process. The contributions of the two Hamiltonians are controlled by the annealing schedule through weighting functions $A(s)$ and $B(s)$, where $s \in [0, 1]$ is the normalized annealing time.

$$H = -\frac{A(s)}{2} \left(\sum_i^{|V|} \hat{\sigma}_x^{(i)} \right) + \frac{B(s)}{2} \left(\sum_i^{|V|} h_i \hat{\sigma}_z^{(i)} + \sum_{i>j} J_{ij} \hat{\sigma}_z^{(i)} \hat{\sigma}_z^{(j)} \right) \quad (1)$$

2.3 Grover

As QAOA and QA are heuristic algorithms, they don't have probabilistic guarantees for finding optimal solutions. In contrast, Grover's algorithm [3] is an exact quantum method.

Grover's algorithm can be used to search a space of $N = 2^n$ elements in $O(\sqrt{N})$ oracle evaluations. Each iteration applies an oracle that phase-marks valid states followed by the Grover diffusion operator, amplifying the probability of measuring marked solutions.

3 Adaptation for cliques

3.1 Clique search problem QUBO mapping

We implemented QUBO formulations for the `MaxClique` optimization problem and the `KClique` decision problem. Both formulations are based on the observation that a clique in a graph G corresponds to an independent set in its complement graph $\bar{G}$. To enforce clique constraints in G , we add penalty terms for each edge in $\bar{G}$ as independence constraints for the independent set problem. For `MaxClique`, the objective is to maximize the number of selected vertices while penalizing violations of the independence constraint (Equation (2)). For `KClique`, however, the clique size constraint enforces the selection of exactly k vertices (Equation (3)).

$$H_{max} = -A \sum_{v \in V(\bar{G})} x_v + B \sum_{(u,v) \in E(\bar{G})} x_u x_v \quad (2)$$

$$H_k = A \left(k - \sum_{v \in V(\bar{G})} x_v \right)^2 + B \sum_{(u,v) \in E(\bar{G})} x_u x_v \quad (3)$$

Both QAOA and QA used the same formulations. Constant A was always set to 1, while B was determined experimentally for both problems and methods separately.

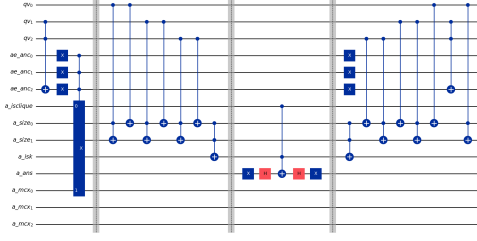


Figure 1: Oracle circuit for checking if a subset of vertices is a k -clique on a line graph with 3 vertices.

	QAOA	QA
KClique	$0.3k - 0.15$	k
MaxClique	1.1	4

Table 1: Best B parameter values for both methods and problems.

3.2 Clique search with Grover

For Grover's algorithm we implemented an oracle circuit for solving the `KClique` problem only as for the `MaxClique` you can simply run the circuit multiple times with increasing k .

The oracle design was inspired by [6]: the first half checks for clique violations by ensuring no two unconnected vertices are chosen together, while the second part checks if the clique size is exactly k (instead of greater than k as in [6] to produce a shallower circuit). A graph's structure is embedded in the choices of gate positions, e.g. checking for violations only among disconnected vertices. The a_{isclique} ancilla qubit is flipped to 1 when there are no clique violations between pairs of vertices, while the a_{isk} ancilla qubit is flipped to 1 when the size of the clique is exactly k . When those two ancillas are both 1, we apply an MCX gate to the a_{anc} qubit in the $|-\rangle$ state, marking k -cliques by flipping the phase. Finally, all operations on the ancilla qubits are uncomputed. A visualization of an example circuit is shown in Figure 1.

4 Experiment setup and results

All test instances used randomly generated graphs according to the $G(n, p)$ Erdős-Renyi (ER) model as it is the most commonly used model in other literature on similar topic. For the `KClique` problem we additionally hid a k -clique by uniformly randomly choosing k vertices and fully connecting them to guarantee a solution exists. Unless specified, the injected clique size for the `KClique` problem was determined as one less than the expected largest clique of an ER model according to [7], which could affect clique distribution more in smaller graphs. For the `MaxClique` problem, we used a classical algorithm to determine the size of the largest clique in the generated graph.

For the best B parameter of the QUBO formulation on both problems, we performed a quick search using both methods and on different graph sizes n . While for the `MaxClique` the optimal B value was a constant, for `KClique` it depended on k (Table 1).

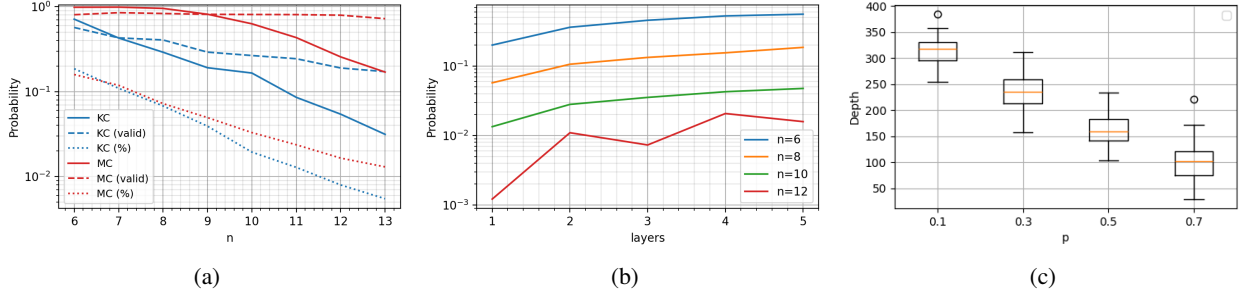


Figure 2: **(a)** The probability of finding cliques at different n . KC and MC represent KClique and MaxClique problems respectively. Dashed lines (valid) represent the probability of a run producing a valid clique, while dotted lines (%) represent the percentage of valid cliques among all vertex subsets of size k . **(b)** Testing how number of QAOA layers affects success probability for different graph sizes (n). More layers appear to improve success probability. **(c)** Transpiled circuit depth for the MaxClique problem as a function of graph density p .

The main metric was the probability that the algorithm finds the correct solution (i.e., a valid clique of the correct size), and we sometimes also plot the probability that the solution represents only a valid clique in cases where the solution has an incorrect number of vertices.

4.1 QAOA

Due to the exponential difficulty of simulating quantum circuits on a classical computer, our tests used graphs with up to 15 nodes. Circuits were simulated locally using the AerSimulator and StatevectorEstimator, with 1000 shots, and optimization level 3 in the generate_preset_pass_manager method. For the noisy backend we switched to FakeBrisbane with its default parameters. We used COBYLA classical optimizer paired with SciPy’s minimize method, with $\text{tol}=1\text{e-}2$. Circuit parameters were initialized uniformly at random in the $[0, 2\pi]$ range.

The mixer Hamiltonian consisted of Pauli-X operator across all qubits as $R_x(\beta_i)$ gates on each layer i , while the problem Hamiltonian was prepared with SparsePauliOp.from_sparse_list from the QUBO formulations (2 and 3). The final circuit was generated using Qiskit’s QAOAAnsatz class.

The experiments tested different graph sizes, densities, target clique sizes k , numbers of QAOA layers, runs per graph, circuit depths produced, and the required number of optimiser function evaluations.

As expected, the difficulty of both KClique and MaxClique scales exponentially in the size of the problem at a fixed number of iterations as shown in Figure 2a although the difficulty of MaxClique seems to start dropping slowly at first, possibly due to having edges and triangles as their largest cliques which are very common subgraph structures. The success probability seems to scale proportionally with the percentage of possible correct solutions. The success probability of the MaxClique problem is higher, possibly because valid cliques are more frequently sampled across the tested problem sizes. Increasing the number of layers also monotonically increased success probability (Figure 2b).

Very dense or very sparse graph instances showed higher solution probabilities than those with a density

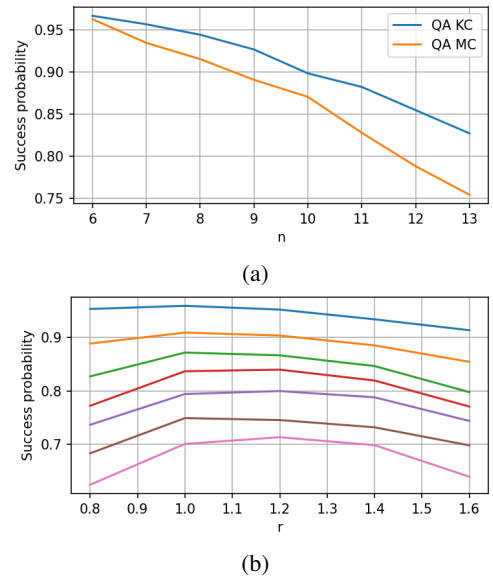


Figure 3: **(a)** Success probability of QA when solving both clique problems, **(b)** and when solving sizes of MaxClique problem at different schedule function exponents r . Lines from top to bottom correspond to $n = 6, 8, 10, 11, 12, 13, 14$.

around 0.4, possibly because they had more correct solutions. With a noisy backend, however, sparse graphs also posed a big problem because of a larger number of Pauli operators in the problem Hamiltonian, which increased the depth of the circuit (Figure 2c).

4.2 Quantum Annealing

QA was simulated using QuTiP’s sesolve method, where we used function $s(t) = (t/T)^r$ for the annealing schedule, and annealing time $T = 10$. The exponent r was tested for different problem sizes. The initial state was $|+\rangle^{\otimes n}$, the ground state of the initial transverse-field Hamiltonian.

With QA, MaxClique is solved less successfully than KClique (Figure 3a). Comparing with QAOA, the drop in success probability seemed slower; however, this might not reflect performance on an actual quantum de-

vice.

Figure 3b shows the best exponent is at around $r = 1.1$ but also suggests larger instances might benefit from larger r .

4.3 Grover’s algorithm

Like QAOA, Grover’s algorithm was simulated locally on the AerSimulator, FakeBrisbane, as well as on a real backend (`ibm_fez`) provided by IBM Quantum Platform¹.

Even for small instances, the produced circuits had substantial depth and the amount of noise was close to covering the correct solutions (Figure 4a). Denser graphs produce shallower circuits since there are fewer clique violation checks on disconnected vertices. For example, a 3-node graph with 0 edges produced a depth of 183, while a graph with 3 edges produced a depth of 142 (for one iteration). In theory, for the 3-vertex, fully connected graph, the ideal number of Grover iterations would be at around 2, but because running deeper circuits would accumulate too much noise, we decided to perform all tests using 1 iteration only.

4.4 Resource consumption

QAOA and QA both consume one (logical) qubit per graph vertex, while for Grover’s algorithm we require $O(n^2)$ qubits due to having one edge ancilla qubit for each vertex pair. Although the number of gates is supposed to scale similarly, measurements suggest circuit depth scales rather linearly thanks to parallel gate execution (Figure 4b). Higher depth and qubit count make Grover’s algorithm challenging to run on current quantum hardware.

5 Conclusion and Future Work

We implemented and evaluated QAOA, QA, and Grover’s algorithm for solving the `KClique` and `MaxClique` problems. While all three methods successfully solved small instances, their scalability on current quantum hardware is primarily limited by circuit depth (especially for Grover’s algorithm) and available qubits. Since Grover’s algorithm was tested on only a 3-node graph, we cannot determine the scalability of our implementation.

Unfortunately, we were unable to test both QAOA and QA on real quantum hardware for a more realistic comparison. Using only ER graphs in our experiments also limits the generalizability of our results to other graph types. QA could be further tested for annealing schedule functions and .

Future work could improve QAOA by replacing the default initial state and mixer with problem-specific alternatives, such as Dicke-state preparation or a Quantum Alternating Operator Ansatz [8], restricting the search to valid solutions. For QA, analyzing the minimum spectral gap and exploring modified Hamiltonians such as trigger Hamiltonians [9] could provide further insight into instance difficulty. Finally, designing shallower oracle cir-

¹<https://quantum.cloud.ibm.com/>

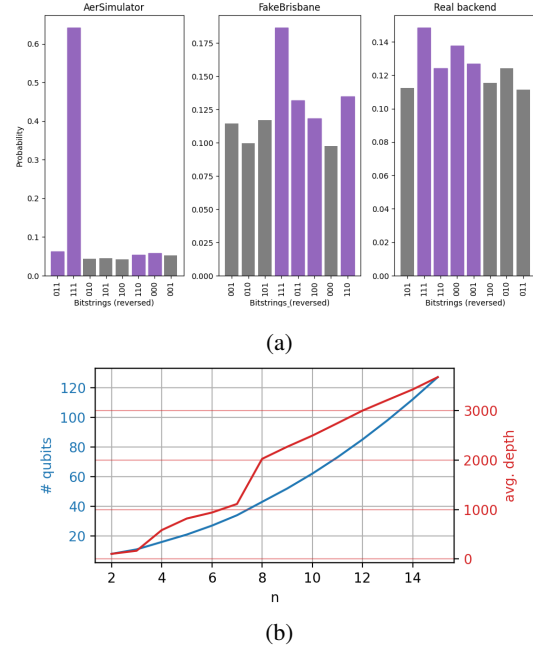


Figure 4: (a) Probabilities of measuring each bitstring at the end of 1 iteration of Grover’s algorithm for solving the `KClique` problem on different backends. The graph we used here was fully connected with 3 vertices with exactly 1 correct solution 111 ($k = 3$). (b) Number of qubits used for Grover’s algorithm and average transpiled circuit depth for $G(n, 0.5)$ graphs.

cuits could improve the practical scalability of Grover’s algorithm.

References

- [1] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman and Company, 1979. ISBN: 0-7167-1045-5.
- [2] E. Farhi et al. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: 1411.4028 [quant-ph].
- [3] L. K. Grover. *A fast quantum mechanical algorithm for database search*. 1996. arXiv: quant-ph/9605043 [quant-ph].
- [4] K. Blekos et al. “A review on Quantum Approximate Optimization Algorithm and its variants”. In: *Physics Reports* 1068 (2024). A review on Quantum Approximate Optimization Algorithm and its variants, pp. 1–66. ISSN: 0370-1573.
- [5] M. Sharma and H. C. Lau. *A Comparative Study of Quantum Optimization Techniques for Solving Combinatorial Optimization Benchmark Problems*. 2025. arXiv: 2503.12121 [quant-ph].
- [6] A. Haverly and S. López. “Implementation of Grover’s Algorithm to Solve the Maximum Clique Problem”. In: *2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 2021, pp. 441–446.
- [7] W. Yang et al. *Lecture 2: Largest Clique in a Random Graph*. 2018.
- [8] S. Hadfield et al. “From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz”. In: *Algorithms* 12.2 (Feb. 2019), p. 34. ISSN: 1999-4893.
- [9] V. Mehta et al. “Quantum annealing with trigger Hamiltonians: Application to 2-satisfiability and nonstoquastic problems”. In: *Physical Review A* 104.3 (2021). ISSN: 2469-9934.